从结对编程到智能体编排

第06讲 · 2026年9月17日

layout: default

今天

三个部分

1. 项目：`ai-assistant`
2. 与模型协作
3. 从编辑块到工具调用

layout: section

项目

`ai-assistant`，10月6日截止

layout: default

项目构建

你自己的结对编程伙伴

- 一个关于你选择的文件的终端对话
- 模型提出编辑，你批准
- 每个批准的变更变成一个提交

Aider，你已经使用了三周的工具。现在你写一个。

layout: default

SPEC.md 第3节

工作人员提供什么和你决定什么

工作人员提供	你决定
合同：7个特性	模块、接口和状态
一个 starter 循环，`bin/assistant`，`bin/test`	估计器、修剪策略、文件分隔符
`ENDPOINT.md`，以及 `SPEC.md` §5 中的检查	错误消息和提交所有权
	你的测试

layout: default

README.md

工作顺序

1. 阅读数据包，`EXAMPLES.md` 优先
2. 设计和图，冷审查
3. 增量，每个规格说明先写
4. 现场证据、使用、反思

设计在应用程序代码之前提交。

layout: default

RUBRIC.md

项目构建如何评分

一半	分数
规格说明和设计	50
实现和测试	50

- 代码之前的规格说明，从提交顺序读取
- 没有隐藏的测试或要求
- 没有痛苦分

layout: default

日期

日期和交付物

时间	事项
这节课之后	数据包，在你的代码库中
任何应用程序代码之前	`design/SYSTEM.md` 和图表
9月24日周四，晚上	Hackathon 1，单独的
10月6日周二，晚上11:59	项目构建截止

一个代码库，整个学期。提交你的 Aider 历史。

layout: section

与模型协作

理解它写什么和它说什么

layout: default

你现在的工作

谁理解代码

- 模型写大多数行
- 你拥有决策和结果
- 在合适层级上的理解

layout: default

上下文存在于两个地方

你脑海中记住的

- 设计以及为什么是这样
- 你在构建中的位置
- 什么实际被验证了
- 模型只声称了什么

模型在会话之间不保留这些中的任何一个。

layout: default

困难部分

在合适距离上的判断

距离	它看起来像什么
太近	阅读它写的每一行
太远	接受你从未检查过的内容
恰到好处	检查合同和结果

不是每一行。每一个决策。

layout: statement

工作关系需要*共享名称*。

layout: default

你定义的一个构造

参考点

名称	它指向什么
`F6`	预览和应用编辑
`design/SYSTEM.md` §6	你的安全决策在哪里
`specs/03-preview.md`	交付它的增量
`test_stale_preview_refused`	什么检查它

一个短的、稳定的名称，你、模型和工作人员以相同的方式解析它。

layout: default

从规划这节课

会话中的决策编码

model: decisions before I write the slides
Code
Decision
D1
The live demo is Claude Code in a repository
D2
The approval layer moves out of this lecture
D3
READ and RUN use made-up markers, not F5's

layout: default

回报

用代码回答

you
D1 yes. D3 yes. D4 no, do READ live. D5 yes.
model
D1, D3, D5 kept.
D4 changed: READ is live, RUN stays on slides.
D2 still open. Keep it out?
you
D2 yes.
十八个决策中的五个，在两行中。

layout: default

DESIGN-GUIDE.md 第1节

数据包留给你的命名

- SPEC 命名其7个特性
- 你命名你的验收标准
- 你的每个名称都 traceable 到一个特性

"重复 SPEC.md 第4节什么也得不到。" RUBRIC.md

layout: default

人类对模型

按名称提示

按意译

```
fix the approval stuff so
it's safer
```

按名称

```
specs/03, task 2 only.
Deliver F6-3.
Make test_stale_preview_refused
pass. Edit preview.py only.
```

layout: default

模型对人类

用你的名称要求回答

```
Before I review: which criteria does this change deliver,
what did you run, and what did you not verify?
```

```
Delivers     F6-3
Touched      assistant/preview.py
Ran          nothing
Unverified   F6-3 against a real target repository
```

layout: default

人类对人类，人类对工具链

一行描述项目状态

```
specs/03 · F6-3 done · F6-4 red: test_write_failure_restores
```

- 给工作人员或邻居的消息
- 新会话的第一行
- 提交消息

layout: default

在哪里失效

名称如何失效

- 意译替换了 ID
- 模型发明了一个 ID
- 命名的文件不在上下文中
- 名称在构建中途改变了含义

layout: default

现场演示

代码库中的 Claude Code

- 按参考点命名任务
- 用这些名称要求报告
- 检查每个声明

layout: default

五分钟与邻居

阅读模型输出的其他方式

参考点是一种构造。

命名**一个其他习惯或构造**，使模型的输出更容易理解，或以其他方式促进沟通。

每人一行。我们将收集五个。

layout: section

从编辑块到工具调用

成长你的程序拥有的一个命令

layout: default

SPEC.md F5

你的程序拥有的一个命令

```
greet.py
<<<<<<< SEARCH
print("Hello")
=======
print("Hi")
>>>>>>> REPLACE
```

来自模型的文本。你的代码解析它并行动，在是的时候。

layout: default

结对编程伙伴卡壳的地方

当模型需要一个文件时

```
you      why does greet() print twice?
model    I can't tell without main.py, which calls greet().
you      /add main.py
you      ok, look again
```

模型知道它需要什么。只有你能获取它。

layout: default

问题

为什么不让它读取

- 它已经命名了它需要的文件
- 读取不会改变磁盘上的任何东西
- 你只是转发而已

所以给它一个询问的方式。

layout: default

添加一个命令

一个 READ 命令

```
I need to see the tests before I change anything.
<<<<<<< READ
tests/test_cli.py
>>>>>>> END
```

你的代码读取文件并将其内容发回。

layout: default

添加另一个

一个 RUN 命令

```
Let me check the suite first.
<<<<<<< RUN
bin/test
>>>>>>> END
```

你的代码运行它并将其输出发回。

layout: default

工具链成长

三个命令的分发器

```
for block in parse_blocks(reply):
    if block.kind == "EDIT":
        result = propose_edit(block.path, block.search, block.replace)
    elif block.kind == "READ":
        result = read_file(block.path)
    elif block.kind == "RUN":
        result = run_command(block.command)
    messages.append({"role": "user", "content": result})
```

layout: default

模式

每个命令的一种形态

块	名称	参数
`SEARCH/REPLACE`	`edit_file`	path, search, replace
`READ`	`read_file`	path
`RUN`	`run_command`	command

一个名称加参数。那就是一个*工具调用*。

layout: default

函数调用

工具调用的形态

```
{"tool_calls": [
  {"name": "read_file",   "arguments": {"path": "tests/test_cli.py"}},
  {"name": "run_command", "arguments": {"command": "bin/test"}},
  {"name": "edit_file",   "arguments": {"path": "greet.py", "search": "...", "replace": "..."}}
]}
```

同样的三个命令，无需自定义格式即可解析。

layout: statement

模型从不运行任何东西。它*询问*。

layout: default

谁驱动

谁发回结果

```
you     why does bin/test fail?
model   READ tests/test_cli.py
you     (contents) keep going
model   RUN bin/test
you     (output) keep going
model   SEARCH/REPLACE greet.py
```

每一个"继续"都是你，作为循环。

layout: default

将人自动化

选择其步骤的循环

```
while True:
    reply = model.call(messages, tools=TOOLS)
    if not reply.tool_calls:
        return reply.text           # the model is done
    for call in reply.tool_calls:
        result = execute(call)      # your code, your rules
        messages.append(result)
```

layout: default

循环改变了什么

循环前后

- 模型仍然只写文本
- 你的代码仍然执行每一个动作
- 模型现在选择下一步

只有最后一行改变了。

layout: default

还剩下什么

循环仍然需要什么

- 迭代和令牌的上限
- 重复调用的检测器
- 一个干净的循环中段中断
- `run_command` 之前的批准

撤销覆盖文件。它不覆盖 shell。

layout: default

接下来做什么

这周和接下来两周

时间	事项
今晚	阅读 `EXAMPLES.md`，然后 SPEC
任何应用程序代码之前	设计、图表、冷审查
9月24日周四，晚上	Hackathon 1
10月6日周二，晚上11:59	项目构建截止

layout: end